UCAC-ICAM — Année Académique 2025–2026
Groupe de Recherche : Groupe 1

PLAN COMPLET DE CONCEPTION ET DE RÉALISATION

Système de Localisation Indoor pour Opérateur de Maintenance Industrielle

Approche WiSLAM / Visual SLAM / FootSLAM — Application Mobile

Équipe : Rosilia MODJO · Nadia TCHUENTE · Joseph NKOULOU
Encadrants : Aloys NGUEPI & Humphrey OJONG
Mai 2026

0. Résumé Exécutif

Objectif du document

Ce document constitue le plan complet de conception et de réalisation du projet de recherche portant sur la localisation d'opérateurs de maintenance en atelier industriel. Il couvre l'étude de faisabilité, l'architecture technique, les spécifications fonctionnelles et non-fonctionnelles, le plan d'implémentation, la répartition des tâches entre les trois membres de l'équipe, et le protocole de tests. Il est conçu pour servir de feuille de route opérationnelle à une équipe sans expérience pratique préalable du domaine.

Le projet vise à développer une application mobile Android permettant à un technicien de maintenance d'être localisé automatiquement dans un atelier industriel et guidé vers l'équipement en panne. La solution repose sur une approche hybride combinant :

- Wi-Fi SLAM (WiSLAM) : utilisation des signaux Wi-Fi des répéteurs existants pour la localisation par trilatération et fingerprinting
- Visual SLAM allégé : exploitation de la caméra du smartphone pour enrichir et corriger la localisation
- Centrale inertielle (IMU) : capteurs du téléphone (accéléromètre, gyroscope, magnétomètre) pour le suivi de déplacement pas-à-pas (PDR)

La solution finale est une application mobile Android fonctionnant en deux modes : un mode cartographie (construction de la carte radio et visuelle de l'atelier) et un mode navigation (guidage en temps réel vers l'équipement cible).

1. Étude de Faisabilité

1.1 Analyse du Contexte et des Contraintes

L'atelier industriel présente des contraintes spécifiques qui rendent la localisation particulièrement difficile. Cette section les identifie et évalue la faisabilité de chaque approche technologique dans ce contexte.

1.1.1 Contraintes de l'Environnement Industriel

Contrainte	Impact sur la localisation	Solution envisagée
Surfaces métalliques réfléchissantes	Multipath Wi-Fi, instabilité RSSI, erreurs de trilatération jusqu'à 5 m	RTT plutôt que RSSI seul, fusion avec IMU pour lisser les erreurs
Machines en mouvement	Modification continue de l'environnement radio et visuel	Renouvellement partiel de la carte radio, algorithmes adaptatifs
Absence de GPS fiable	Pas de référence absolue de position	Initialisation manuelle + boucles de fermeture Wi-Fi
Bruit électromagnétique (moteurs, câbles)	Distorsion du champ magnétique, interférence Wi-Fi	Minimiser l'usage du magnétomètre, privilégier Wi-Fi RTT
Ressources limites (tablette / smartphone)	Contraintes CPU/RAM pour algorithmes lourds (SLAM)	Implémentation allégée, nombre de particules réduit

1.1.2 Inventaire du Matériel Disponible

Équipement	Caractéristiques supposées	Rôle dans le projet
Répéteurs Wi-Fi (N ≥ 3)	Points d'accès Wi-Fi 2.4/5 GHz ; support IEEE 802.11mc (RTT) à vérifier	Ancres de référence pour trilatération et fingerprinting
Caméra smartphone	Caméra arrière standard, résolution ≥ 8 MP, autofocus	Visual SLAM allégé, détection de marqueurs ArUco
IMU smartphone	Accéléromètre + gyroscope + magnétomètre intégrés	PDR (détection de pas, estimation de cap)
Smartphone Android	Android 10+ requis (API Wi-Fi RTT : API 28+)	Plateforme principale de l'application

1.2 Analyse de Faisabilité par Approche

1.2.1 Wi-Fi RSSI Fingerprinting

Principe : collecter des empreintes de signal (vecteurs RSSI) à des points de référence connus, puis comparer les mesures en temps réel à cette base pour estimer la position.

- Faisabilité : HAUTE. Technologie mature, ne nécessite pas de support hardware spécifique
- Précision attendue : 1 à 3 mètres en environnement standard, potentiellement dégradée (2–5 m) en atelier
- Contrainte majeure : la phase de cartographie (offline) est longue et doit être refaite si l'environnement change
- Verdict : faisable comme solution de base (Solution 2), mais insuffisante seule pour navigation fine

1.2.2 Wi-Fi RTT (Round Trip Time) + SLAM

Principe : exploiter la mesure de distance directe AP-appareil via IEEE 802.11mc, sans connaissance préalable des positions des AP, dans un framework de graphe de poses SLAM.

- Faisabilité : MOYENNE. Dépend du support RTT par les répéteurs ET par le smartphone
- Action préalable critique : vérifier via l'application Android 'Wi-Fi RTT Scanner' que les répéteurs répondent aux requêtes RTT
- Précision attendue : 0.1 à 0.5 m en résidentiel (rWiFiSLAM), probablement 0.3–1 m en industriel
- Verdict : solution prioritaire (Solution 1) si le matériel le supporte

1.2.3 Visual SLAM via Caméra

Principe : utiliser la caméra pour identifier des points d'intérêt visuels (features) ou des marqueurs ArUco placés dans l'atelier, afin de corriger et enrichir la localisation Wi-Fi.

- Faisabilité : HAUTE en mode hybride (marqueurs ArUco), MOYENNE en mode features (ORB-SLAM)
- Recommandation : commencer par marqueurs ArUco (plus simples, plus robustes), ajouter features si le temps le permet
- Précision attendue : positionnement sub-décimétrique à proximité des marqueurs
- Verdict : complément idéal au Wi-Fi, particulièrement utile près des équipements cibles

1.2.4 IMU / PDR (Pedestrian Dead Reckoning)

Principe : utiliser l'accéléromètre pour détecter les pas et le gyroscope pour suivre le cap, afin d'estimer le déplacement de l'opérateur entre deux mises à jour Wi-Fi.

- Faisabilité : TRÈS HAUTE. Tous les smartphones modernes possèdent les capteurs requis
- Contrainte : dérive cumulative — l'erreur augmente de 2 à 5 % de la distance parcourue
- Rôle : pont entre les mises à jour Wi-Fi (lentes) pour assurer un affichage fluide de la position
- Verdict : composant indispensable et facile à implémenter

1.3 Décision Architecturale

Architecture retenue : Hybride Wi-Fi + IMU + Vision

Après analyse, l'architecture retenue combine trois couches complémentaires : (1) Wi-Fi comme source principale de localisation absolue (RTT si disponible, fingerprinting sinon), (2) IMU pour le suivi continu du déplacement entre les mises à jour Wi-Fi, et (3) caméra pour la correction ponctuelle via marqueurs ArUco et l'identification visuelle des équipements. Cette combinaison offre le meilleur rapport précision / faisabilité / coût pour le contexte industriel ciblé.

2. Architecture Système

2.1 Vue d'Ensemble

Le système se décompose en trois grandes parties : l'application mobile Android (client), un serveur léger de gestion des alertes de maintenance, et l'infrastructure Wi-Fi de l'atelier (répéteurs existants).

Couche Application Mobile	Couche Serveur	Couche Infrastructure
Interface utilisateur (carte 2D + guidage), moteur de localisation hybride, module PDR, module caméra ArUco, module Wi-Fi scan	API REST légère, base de données des équipements et alertes, carte de l'atelier (positions des AP et équipements)	Répéteurs Wi-Fi (ancres), marqueurs ArUco dans l'atelier (optionnels mais recommandés)

2.2 Architecture de l'Application Mobile

2.2.1 Modules Principaux

L'application Android est organisée en modules découplés, chacun responsable d'une fonction précise :

Module	Description	Technologie principale
WiFiScanner	Scan périodique des AP : RSSI et RTT (si supporté), construction du vecteur d'observation	Android WifiManager API, WifiRttManager API
IMUProcessor	Détection de pas (threshold sur norme acc.), estimation de longueur de pas (Weinberg), suivi de cap (gyro + boussole), calcul PDR	SensorManager Android, algorithme Weinberg
CameraModule	Détection et décodage de marqueurs ArUco, estimation de pose 3D (solvePnP), déclenchement sur demande ou automatique	OpenCV Android SDK, ArUco module
LocalizationEngine	Fusion des données Wi-Fi + IMU + caméra, filtre particulaire ou graphe de poses, estimation de position courante	Java/Kotlin custom, bibliothèque de filtres
MapRenderer	Affichage de la carte 2D de l'atelier, position de l'opérateur, équipement cible, chemin guidé	Canvas Android ou bibliothèque SVG
NavigationEngine	Calcul du chemin optimal vers l'équipement cible (A* ou Dijkstra sur grille), instructions de guidage (virage à gauche, 10m tout droit…)	Algorithme A* implémenté en Java/Kotlin
ServerClient	Communication avec le serveur : récupération des alertes de maintenance, envoi de la position (optionnel), téléchargement de la carte	Retrofit / OkHttp, REST JSON

2.3 Pipeline de Localisation

Le pipeline de localisation fonctionne en boucle continue à des fréquences différentes selon la source :

- IMU : 50–100 Hz — mise à jour PDR très fréquente pour un affichage fluide
- Wi-Fi scan : 0.2–1 Hz — mise à jour de position absolue pour corriger la dérive PDR
- Caméra ArUco : à la demande ou automatique quand un marqueur est détecté

La fusion suit la logique suivante à chaque cycle :

1. Le module IMU accumule les pas depuis la dernière mise à jour Wi-Fi et prédit la position courante (dead reckoning)
2. À chaque scan Wi-Fi, le moteur de localisation compare les observations aux données de référence et génère une estimation de position absolue
3. Le filtre de fusion (filtre particulaire simplifié à 100–500 particules) combine les deux estimations en pondérant selon la qualité du signal
4. Si un marqueur ArUco est détecté, sa pose estimée est injectée comme contrainte forte dans le filtre
5. La position finale est transmise au MapRenderer pour affichage et au NavigationEngine pour le calcul du guidage

2.4 Modes de Fonctionnement

Mode 1 : Cartographie (Phase Offline)

L'opérateur parcourt l'atelier en activant le mode cartographie. L'application collecte des mesures Wi-Fi (RSSI + RTT) à des points d'intérêt réguliers et construit la carte radio. Des marqueurs ArUco sont placés manuellement à des positions connues (mesurées à la règle). Cette phase est réalisée une seule fois, puis mise à jour si nécessaire.

Mode 2 : Navigation (Phase Online)

L'opérateur reçoit une alerte de maintenance sur son téléphone. Il ouvre l'application, qui se localise automatiquement (initialisation par Wi-Fi fingerprinting ou scan d'un marqueur ArUco). La carte s'affiche avec sa position et l'équipement cible mis en évidence. Des instructions de guidage sont affichées en temps réel.

3. Spécifications Techniques

3.1 Spécifications Fonctionnelles

3.1.1 Fonctions Obligatoires (MVP)

- SF01 — Scan Wi-Fi : l'application doit scanner les AP disponibles et mesurer le RSSI de chacun à une fréquence minimale de 0.5 Hz
- SF02 — Fingerprinting : l'application doit comparer le vecteur RSSI courant à la base de données et retourner la position la plus probable par algorithme k-NN (k=3)
- SF03 — PDR : l'application doit détecter les pas de l'opérateur et estimer sa position par dead reckoning avec une erreur de dérive inférieure à 5 % sur 50 m
- SF04 — Fusion Wi-Fi + IMU : les deux estimations doivent être fusionnées pour produire une position unifiée affichée en temps réel
- SF05 — Affichage carte : la position de l'opérateur doit être affichée sur une carte 2D de l'atelier avec rafraîchissement ≥ 2 Hz
- SF06 — Guidage : l'application doit calculer et afficher le chemin vers l'équipement cible sélectionné, avec mise à jour à chaque déplacement
- SF07 — Mode cartographie : l'application doit permettre la collecte de données de fingerprinting et l'enregistrement de la carte radio

3.1.2 Fonctions Avancées (si temps disponible)

- SF08 — Wi-Fi RTT : si les répéteurs supportent IEEE 802.11mc, utiliser les mesures RTT à la place ou en complément du RSSI
- SF09 — Détection ArUco : scanner en continu le flux caméra pour détecter des marqueurs ArUco et corriger la position
- SF10 — Serveur d'alertes : connexion à un serveur léger pour recevoir les alertes de maintenance avec l'identifiant de l'équipement
- SF11 — Historique de trajectoire : enregistrement et visualisation du trajet parcouru
- SF12 — Graphe de poses SLAM : remplacement du filtre particulaire simple par un graphe de poses avec fermeture de boucles Wi-Fi

3.2 Spécifications Non-Fonctionnelles

Critère	Exigence	Méthode de vérification
Précision (MVP)	Erreur de localisation médiane ≤ 3 m en conditions normales	Mesure sur 20 points de test, calcul RMSE
Précision (cible)	Erreur de localisation médiane ≤ 1.5 m	Idem, avec fusion Wi-Fi RTT + ArUco
Latence affichage	Position mise à jour en moins de 500 ms après chaque pas	Mesure du délai IMU → affichage
Dérive PDR	< 5 % de la distance parcourue avant correction Wi-Fi	Test sur trajectoire de 50 m en ligne droite
Performance CPU	Utilisation CPU < 60 % en mode navigation normale	Profiler Android Studio
Autonomie batterie	Durée de fonctionnement ≥ 2 h en utilisation continue	Test de décharge sur 2 h de navigation
Compatibilité	Android 10 (API 29) minimum ; Android 12 recommandé	Tests sur 2 appareils différents

Critère	Exigence	Méthode de vérification
Robustesse	Pas de crash en 30 min de navigation continue	Test de stabilité prolongé

3.3 Stack Technologique

Composant	Technologie retenue	Justification
Langage mobile	Pas encore choisie (Android)	Langage officiel Android, excellent support API capteurs
IDE	Android Studio	Environnement standard Android, profiler intégré
Vision par ordinateur	OpenCV 4.x Android SDK	Bibliothèque de référence, support ArUco natif
Wi-Fi RTT	WifiRttManager (API 28+)	API Android officielle pour IEEE 802.11mc
Affichage carte	Canvas Android (custom View)	Contrôle total du rendu, pas de dépendance externe
Serveur (optionnel)	Python Flask + SQLite	Léger, rapide à déployer, facilement testable
Communication client-serveur	REST HTTP + JSON	Standard industry, bibliothèque mature pour Android
Gestion de versions	Git + GitHub	Collaboration entre 3 membres, historique complet
Tests unitaires	JUnit 5 + Mockito	Framework de test standard Android / Java

4. Plan d'Implémentation Détaillé

4.1 Vue d'Ensemble des Phases

Le projet est structuré en 5 phases successives, d'une durée totale estimée à 14 semaines. Chaque phase produit un livrable concret et testable.

Phase	Intitulé	Livrable
Phase 0	Préparation & Vérification Matériel	Rapport de faisabilité matériel
Phase 1	Socle Applicatif & Collecte de Données	App basique + dataset Wi-Fi
Phase 2	Localisation Wi-Fi + IMU (Solution 2 MVP)	Localisation par fingerprinting fonctionnelle
Phase 3	Fusion Avancée & Visual SLAM (Solution 1)	Localisation hybride Wi-Fi RTT + ArUco + IMU
Phase 4	Interface & Guidage Navigation	Application complète avec guidage
Phase 5	Tests, Évaluation & Documentation	Rapport de tests, documentation finale

4.2 Phase 0 — Préparation & Vérification Matériel

Objectif de la phase

Vérifier que le matériel disponible supporte les technologies envisagées, mettre en place l'environnement de développement, et décider définitivement de la stratégie Wi-Fi (RTT ou RSSI seul).

4.2.1 Tâches Détaillées

- **T0.1 — Test Wi-Fi RTT : installer l'application 'Wi-Fi RTT Ranging Sample' (disponible sur GitHub officiel Android) et tester si les répéteurs répondent aux requêtes RTT. DÉCISION CRITIQUE : si RTT supporté → Solution 1 prioritaire ; sinon → Solution 2 uniquement**
- T0.2 — Test OpenCV : créer un projet Android minimal avec OpenCV et afficher le flux caméra avec détection d'un marqueur ArUco imprimé sur papier A4
- T0.3 — Configuration Git : créer le dépôt GitHub avec la structure de projet, les branches par membre, et les règles de commit
- T0.4 — Configuration Android Studio : installer SDK, NDK (pour OpenCV), et configurer le projet Gradle
- T0.5 — Plan de l'espace test : choisir l'espace de test (salle, couloir, ou atelier), mesurer ses dimensions, noter les positions des répéteurs

4.2.2 Livrables Phase 0

- Rapport de 2 pages : résultat test RTT, résultat test ArUco, décision architecturale finale
- Dépôt GitHub configuré avec structure de base
- Plan de l'espace test annoté (positions AP, dimensions)

4.3 Phase 1 — Socle Applicatif & Collecte de Données

Objectif de la phase

Construire les briques fondamentales de l'application : scan Wi-Fi, lecture IMU, affichage basique, et collecte du dataset de fingerprinting qui sera utilisé pour entraîner et tester l'algorithme de localisation.

4.3.1 Module WiFiScanner

- Implémenter le scan Wi-Fi via WifiManager.startScan() avec callback sur SCAN_RESULTS_AVAILABLE_ACTION
- Extraire : BSSID (MAC), SSID, RSSI (dBm), fréquence, et RTT distance si disponible
- Construire le vecteur d'observation : liste ordonnée de RSSI pour les AP connus (valeur par défaut -100 dBm si AP non détecté)
- Paramètre clé : sélectionner les N AP les plus fréquemment détectés (stratégie k = 2/3 du total, inspirée de l'article 6)

4.3.2 Module IMUProcessor

- Lire les capteurs via SensorManager (TYPE_ACCELEROMETER, TYPE_GYROSCOPE, TYPE_ROTATION_VECTOR)
- Algorithme de détection de pas : calculer la norme de l'accélération, détecter les pics au-dessus d'un seuil (typiquement 10–12 m/s²), appliquer un temps mort de 250 ms entre deux pas
- Estimation de longueur de pas : formule de Weinberg — L = K × (amax − amin)^(1/4) où K est un coefficient de calibration (à calibrer par chaque utilisateur sur 10 m mesurés)
- Estimation du cap : utiliser le vecteur de rotation (TYPE_ROTATION_VECTOR) pour obtenir l'azimut en degrés
- Calcul PDR : position(t) = position(t-1) + longueur_pas × (cos(cap), sin(cap))

4.3.3 Collecte du Dataset de Fingerprinting

Cette étape est cruciale pour la Solution 2 et sert de référence pour la Solution 1.

- Définir une grille de points de référence espacés de 1 à 1.5 m dans l'espace test
- À chaque point de référence : collecter 30 scans Wi-Fi successifs (durée ≈ 60 secondes), noter la position GPS intérieure (coordonnées X, Y en mètres depuis l'origine)
- Format de stockage : JSON ou CSV — { position: [x, y], scans: [[rssi_AP1, rssi_AP2, ...], ...] }
- Volume cible : minimum 30 points de référence pour un espace de 10×10 m
- Outil de collecte : développer une interface simple dans l'app pour guider l'opérateur pendant la collecte

4.4 Phase 2 — Localisation Wi-Fi + IMU / Solution 2 MVP

Objectif de la phase

Implémenter la Solution 2 complète et fonctionnelle : fingerprinting Wi-Fi + fusion IMU avec filtre particulaire simplifié. Cette solution sert de baseline et garantit un MVP opérationnel avant d'attaquer les algorithmes plus complexes.

4.4.1 Algorithme de Fingerprinting (k-NN)

- Charger la base de données de fingerprinting en mémoire au démarrage
- À chaque scan Wi-Fi : calculer la distance euclidienne entre le vecteur courant et chaque vecteur de référence
- Retourner la position moyenne des k=3 voisins les plus proches (WKNN : pondérés par l'inverse de la distance)
- Amélioration optionnelle : normaliser les vecteurs RSSI avant calcul de distance

4.4.2 Filtre Particulaire Simplifié

- Initialisation : 200 particules réparties uniformément sur la carte
- Étape de prédiction (IMU) : déplacer chaque particule selon le déplacement PDR estimé + bruit gaussien ($\sigma = 0.2$ m)
- Étape de mise à jour (Wi-Fi) : calculer le poids de chaque particule comme $\exp(-d^2/2\sigma^2)$ où d est la distance entre la position de la particule et l'estimation fingerprinting
- Rééchantillonnage : algorithme de roulette biaisée (systematic resampling) quand l'ESS (Effective Sample Size) tombe en-dessous de N/2
- Estimation finale : moyenne pondérée des positions des particules

4.4.3 Affichage Basique

- Afficher la carte de l'espace test (image PNG chargée en fond)
- Afficher la position estimée comme un point bleu avec rayon d'incertitude
- Afficher la trajectoire des 30 dernières secondes en pointillés

4.5 Phase 3 — Fusion Avancée & Visual SLAM / Solution 1

Objectif de la phase

Implémenter la Solution 1 basée sur Wi-Fi RTT + IMU dans un graphe de poses, enrichie par la détection de marqueurs ArUco. Cette phase est la plus complexe algorithmiquement et doit être abordée progressivement.

4.5.1 Module Wi-Fi RTT (si supporté)

- Utiliser WifiRttManager.startRanging() pour obtenir des mesures de distance vers les AP RTT-compatibles
- Fréquence de mesure : 1 Hz (limite matérielle courante)
- Filtrage des mesures aberrantes : rejeter les mesures avec variance > seuil ou RSSI < -80 dBm
- Si moins de 3 AP RTT disponibles : basculer automatiquement sur le fingerprinting RSSI

4.5.2 Détection de Marqueurs ArUco

- Intégrer OpenCV avec le module ArUco dans le projet Android (dépendance Gradle)
- Configurer la caméra en mode continu avec traitement des frames à 5–10 Hz (pour limiter la consommation CPU)
- Détecter les marqueurs ArUco dans chaque frame : aruco.detectMarkers()
- Pour chaque marqueur détecté : estimer sa pose 3D par solvePnP (nécessite calibration caméra préalable)
- Convertir la pose du marqueur en position 2D de l'opérateur (en connaissant la position du marqueur dans la carte)

- PRÉPARATION : imprimer 5 à 10 marqueurs ArUco de taille 20×20 cm (générés via https://chev.me/arucogen/) et les placer à des positions connues dans l'atelier (mesurées avec un mètre ruban)

4.5.3 Calibration de la Caméra

Étape préalable indispensable pour la détection ArUco précise :

- Imprimer un damier de calibration (pattern standard OpenCV, 9×6 cases)
- Capturer 20–30 images du damier sous différents angles
- Exécuter calibrateCamera() d'OpenCV pour obtenir les paramètres intrinsèques (focale, point principal, distorsion)
- Stocker ces paramètres dans un fichier de configuration de l'application

4.5.4 Graphe de Poses (version simplifiée)

- Si le temps le permet : implémenter un graphe de poses simple en mémoire
- Chaque nœud du graphe représente une pose estimée (x, y, θ) à un instant t
- Les arêtes représentent des contraintes : déplacement IMU (odométrie) ou correction Wi-Fi/ArUco
- Optimisation par moindres carrés (Gauss-Newton simplifié) déclenchée toutes les 30 secondes ou lors d'une fermeture de boucle
- Note : si trop complexe, maintenir le filtre particulaire et utiliser les corrections ArUco comme observations directes

4.6 Phase 4 — Interface & Guidage

4.6.1 Interface Utilisateur Complète

- Écran principal : carte 2D de l'atelier avec position de l'opérateur (point animé), équipement cible (icône rouge), chemin calculé (ligne verte), indicateur de précision
- Écran d'alerte : réception d'une alerte de maintenance avec nom et description de l'équipement, bouton 'M'y rendre'
- Écran de configuration : paramètres du filtre, mode de localisation actif, informations de débogage (RSSI, précision)
- Indicateurs en temps réel : précision estimée (cercle autour du point de position), qualité du signal Wi-Fi (barres), mode de localisation actif

4.6.2 Moteur de Navigation

- Représenter la carte comme une grille 2D avec cases libres et obstacles (murs, machines)
- Algorithme A* pour calculer le chemin optimal entre la position courante et l'équipement cible
- Mise à jour du chemin à chaque changement de position significatif (> 0.5 m de déviation par rapport au chemin prévu)
- Instructions textuelles : calculer l'angle entre la direction courante (cap IMU) et le prochain segment du chemin pour générer 'Tournez à gauche dans 5 m', 'Continuez tout droit', etc.

4.7 Phase 5 — Tests, Évaluation & Documentation

Cette phase est détaillée dans la Section 5 (Protocole de Tests). La documentation finale inclut : rapport de tests complet, manuel utilisateur, documentation technique du code (commentaires Kotlin + README), et présentation de soutenance.

5. Protocole de Tests et d'Évaluation

5.1 Métriques d'Évaluation

Métrique	Définition	Seuil d'acceptation
RMSE (m)	Racine de l'erreur quadratique moyenne sur N points de test	< 3 m (MVP), < 1.5 m (cible)
Erreur médiane (m)	Erreur au 50e percentile de la distribution des erreurs	< 2 m (MVP), < 1 m (cible)
Erreur P90 (m)	90e percentile : l'erreur est inférieure à cette valeur dans 90 % des cas	< 5 m (MVP), < 2.5 m (cible)
Dérive PDR (%)	Erreur de position après marche sans correction Wi-Fi / distance parcourue	< 5 % sur 50 m
Latence (ms)	Délai entre le début d'un déplacement et la mise à jour de la position affichée	< 500 ms
Précision ArUco (cm)	Erreur de position lors d'une correction par marqueur ArUco	< 30 cm à 2 m du marqueur
Taux d'échec (%)	Pourcentage de positions où l'erreur dépasse 5 m (positions aberrantes)	< 10 %

5.2 Procédure de Test de Précision

5.2.1 Préparation de la Vérité Terrain

- Sélectionner 20 points de test répartis uniformément dans l'espace test (différents des points de calibration du fingerprinting)
- Mesurer précisément la position de chaque point de test (coordonnées X, Y en mètres depuis l'origine) avec un mètre ruban
- Marquer physiquement ces points au sol (croix au scotch ou repère au sol)

5.2.2 Procédure de Mesure

- L'opérateur se positionne sur chaque point de test et reste immobile 30 secondes
- Enregistrer 10 estimations de position consécutives par point
- Calculer l'erreur euclidienne : $e = \sqrt{(x_estimé - x_réel)^2 + (y_estimé - y_réel)^2}$
- Répéter la procédure à 3 moments différents de la journée pour évaluer la robustesse temporelle
- Tester également en conditions dégradées : présence de 2–3 personnes supplémentaires, machine en fonctionnement si disponible

5.2.3 Test de Navigation Complète

- Définir 5 scénarios de navigation : départ → équipement cible avec différentes distances (5 m, 10 m, 20 m, 30 m, 50 m)
- Chronométrer le temps depuis l'alerte jusqu'à l'arrivée à l'équipement (< 2 m de l'équipement)

- Comparer avec le temps de navigation sans système (opérateur guidé seulement par verbal)
- Évaluer subjectivement l'utilisabilité : l'opérateur trouve-t-il l'interface intuitive ? Les instructions sont-elles claires ?

5.3 Comparaison des Deux Solutions

Les deux solutions (Solution 1 : Wi-Fi RTT/SLAM, Solution 2 : Fingerprinting + Particules) doivent être testées dans les mêmes conditions avec les mêmes 20 points de test. Le tableau comparatif suivant sera rempli lors des tests :

Critère	Solution 1 (Wi-Fi RTT + SLAM)	Solution 2 (Fingerprinting + PF)
RMSE (m)	À mesurer	À mesurer
Erreur médiane (m)	À mesurer	À mesurer
Taux d'échec (%)	À mesurer	À mesurer
Temps de démarrage (s)	À mesurer	À mesurer
Effort de cartographie	Faible (pas d'offline)	Élevé (collecte fingerprints)
Robustesse environnement changeant	Bonne (pas de carte radio figée)	Mauvaise (carte radio stale)
Complexité d'implémentation	Haute	Moyenne

6. Répartition des Tâches par Membre

Principe de répartition

La répartition est organisée selon les spécialités et les phases du projet. Chaque membre est responsable principal d'un axe technique, mais tous contribuent à la phase de tests et à la documentation. Des sessions de synchronisation hebdomadaires sont recommandées pour maintenir la cohérence entre les modules.

6.1 Rosilia MODJO — Responsable Localisation Wi-Fi

Axe principal : Module Wi-Fi & Algorithmes de Localisation

Phase	Tâches	Livrables attendus
Phase 0	T0.1 : test Wi-Fi RTT sur les répéteurs. T0.5 : plan de l'espace test, positions des AP	Rapport test RTT (2 pages), plan annoté de l'atelier
Phase 1	Implémenter WiFiScanner complet (RSSI + RTT). Développer l'outil de collecte de fingerprinting. Piloter la collecte du dataset (30+ points)	Module WiFiScanner testé, dataset fingerprinting complet (JSON/CSV)
Phase 2	Implémenter l'algorithme k-NN (WKNN). Intégrer et tester la localisation fingerprinting seule. Mesurer la précision avant fusion	Module fingerprinting fonctionnel, rapport de précision intermédiaire
Phase 3	Implémenter la localisation par Wi-Fi RTT (trilatération). Si graphe de poses : implémenter la contrainte Wi-Fi. Comparer RTT vs RSSI	Module RTT fonctionnel, rapport de comparaison RTT vs RSSI
Phase 5	Conduire les tests de précision Wi-Fi. Analyser les résultats, calculer RMSE et percentiles. Rédiger la section tests du rapport	Tableau de résultats complet, analyse comparative Solution 1 vs 2

- Compétences à développer : API Android WifiManager/WifiRttManager, algorithme k-NN, trilatération par RSSI
- Ressource principale : Article 1 (Machine Learning Wi-Fi), Article 2 (Fingerprinting overview), Article 3 (Trilatération), Article 5 (rWiFiSLAM)

6.2 Nadia TCHUENTE — Responsable IMU & Fusion

Axe principal : Module IMU/PDR & Moteur de Fusion

Phase	Tâches	Livrables attendus
Phase 0	T0.3 : configurer le dépôt GitHub (branches, règles, CI simple). T0.4 : mettre en place le projet Android Studio de base	Dépôt GitHub opérationnel, projet Android compilable par tous
Phase 1	Implémenter IMUProcessor complet (SensorManager, détection de pas Weinberg, calcul de cap). Calibrer avec des tests de marche mesurée (10, 20, 50 m)	Module IMU fonctionnel, fichier de calibration K, rapport dérive PDR

Phase	Tâches	Livrables attendus
Phase 2	Implémenter le filtre particulaire (200 particules). Intégrer Wi-Fi et IMU dans le filtre. Tester la fusion sur des trajectoires simples (couloir 20 m)	Module LocalizationEngine v1 fonctionnel, rapport fusion intermédiaire
Phase 3	Intégrer les corrections ArUco dans le filtre (si Phase 3 avance). Optionnel : implémenter graphe de poses simple. Tests de robustesse du filtre	Module LocalizationEngine v2 avec fusion triple (Wi-Fi + IMU + ArUco)
Phase 5	Tests de dérive PDR seul. Tester la robustesse de la fusion. Rédiger la section technique du rapport (algorithmes de fusion)	Graphiques de dérive, tableaux de résultats de fusion

- Compétences à développer : SensorManager Android, algorithme de Weinberg, filtre particulaire, Kotlin coroutines pour threading
- Ressource principale : Article 4 (IMU/Wi-Fi Particle Filter), Article 6 (Seq2Seq fingerprinting), Article 7 (FootSLAM)

6.3 Joseph NKOULOU — Responsable Vision & Interface

Axe principal : Module Caméra, Interface Utilisateur & Navigation

Phase	Tâches	Livrables attendus
Phase 0	T0.2 : test OpenCV + ArUco sur Android. Calibration caméra initiale sur damier. Impression et placement de 5 marqueurs test	Preuve de concept : démo vidéo ArUco détecté, paramètres caméra calibrés
Phase 1	Développer la vue de carte basique (Canvas Android). Afficher un point sur la carte. Développer l'écran de collecte de fingerprinting (UI guidée)	Carte affichée avec point de position simulé, écran de collecte opérationnel
Phase 2	Afficher la position estimée en temps réel. Afficher la trajectoire. Développer l'écran de configuration/debug	Interface v1 complète avec position temps réel
Phase 3	Implémenter CameraModule complet (détection ArUco, estimation de pose, conversion en position 2D). Implémenter le serveur Flask minimal (alertes). Tester la correction ArUco en conditions réelles	Module ArUco fonctionnel, serveur de test opérationnel
Phase 4	Implémenter le moteur de navigation A*. Développer les instructions de guidage. Finaliser l'interface utilisateur complète (écran alerte + navigation + debug)	Application complète avec guidage navigation fonctionnel
Phase 5	Tests de précision ArUco. Tests de navigation complète (5 scénarios). Rédiger manuel utilisateur. Préparer la présentation de soutenance	Manuel utilisateur, slides de soutenance, vidéo démo

- Compétences à développer : OpenCV Android, CameraX, Canvas Android, algorithme A*, Python Flask

- Ressource principale : Article 5 (rWiFiSLAM — architecture globale), Article 7 (FootSLAM — logique de localisation)

6.4 Calendrier Global et Jalons

Semaine	Activité principale	Rosilia (Wi-Fi)	Nadia (IMU)	Joseph (Vision/UI)
S1	Phase 0 — Préparation	Test RTT + Plan atelier	GitHub + Android Studio	OpenCV + ArUco PoC
S2	Phase 1 — Socle	WiFiScanner	IMUProcessor	Carte basique
S3	Phase 1 — Collecte dataset	Collecte fingerprints	Calibration PDR	UI collecte
S4	Phase 2 — k-NN + PDR	Algorithme k-NN	Filtre particulaire	Affichage position
S5	Phase 2 — Fusion MVP	Intégration Wi-Fi + filtre	Fusion Wi-Fi + IMU	Trajectoire affichée
S6	Phase 2 — Tests MVP	Tests précision S2	Tests robustesse filtre	Interface debug
S7	Jalon MVP — Solution 2 fonctionnelle	—	—	—
S7–8	Phase 3 — Wi-Fi RTT + ArUco	Module RTT	Fusion triple	CameraModule ArUco
S9–10	Phase 3 — Intégration avancée	RTT vs RSSI compare	Graphe de poses (opt.)	Serveur Flask
S11–12	Phase 4 — UI & Navigation	Support tests navigation	Support tests fusion	A* + interface complète
S13–14	Phase 5 — Tests & Docs	Tests Wi-Fi + rapport	Tests fusion + rapport	Manuel + soutenance

7.2 Points de Blocage Fréquents et Solutions

Problème potentiel	Solution
Répéteurs ne supportent pas Wi-Fi RTT	Basculer complètement sur la Solution 2 (fingerprinting). Perte minime : le fingerprinting donne des résultats comparables dans un espace < 200 m²
OpenCV ne compile pas sur Android	Utiliser le SDK précompilé OpenCV Android (pas les sources). Suivre exactement le tutoriel officiel. Utiliser NDK version 21 (LTS, plus compatible)
Filtre particulaire trop lent (lag perceptible)	Réduire à 100 particules. Déplacer le calcul dans un thread séparé (Coroutine Dispatchers.Default). Ne pas mettre à jour l'affichage à chaque iteration mais toutes les 200 ms
Android 12+ bloque les scans Wi-Fi fréquents	Android limite les scans à 4 par 2 minutes en arrière-plan. Garder l'app en avant-plan (wakelock). Utiliser le mode développeur pour désactiver la limite (pour les tests)
Dérive PDR trop importante en atelier	Augmenter la fréquence de correction Wi-Fi. Utiliser TYPE_ROTATION_VECTOR plutôt que le gyroscope seul pour le cap. Calibrer K sur la démarche réelle de l'opérateur
ArUco non détecté dans l'atelier (luminosité)	Augmenter la taille des marqueurs à 30×30 cm. Laminer les marqueurs pour éviter les reflets. Utiliser un éclairage artificiel stable. Tester plusieurs dictionnaires ArUco (DICT_4X4_50 le plus robuste)

8. Analyse des Risques et Plans de Contingence

Risque	Probabilité	Impact	Plan de contingence
Wi-Fi RTT non supporté par les répéteurs	Élevée	Moyen	Utiliser uniquement RSSI fingerprinting (Solution 2). La précision sera de 1–3 m au lieu de sub-mètre, ce qui reste suffisant pour guider l'opérateur à l'échelle d'un atelier
Précision insuffisante en environnement industriel réel	Moyenne	Élevé	Augmenter la densité de marqueurs ArUco près des équipements critiques pour une correction locale de précision. Réduire la granularité de la navigation (guidage à la zone, pas au cm)
Smartphone insuffisamment puissant pour le traitement temps réel	Faible	Élevé	Réduire le nombre de particules (50 au lieu de 200). Désactiver le traitement caméra en continu (déclencher manuellement). Offloader certains calculs au serveur Flask
Dataset fingerprinting difficile à collecter	Faible	Moyen	Réduire la densité des points de référence (2 m au lieu de 1 m). Utiliser seulement les N=5 AP les plus stables. Tester d'abord dans un couloir simple avant de passer à l'atelier complet
Manque de temps pour la Solution 1 (SLAM)	Moyenne	Faible	La Solution 2 (fingerprinting + PF) est un MVP complet et valide scientifiquement. Le SLAM est un bonus. Se concentrer sur un MVP robuste plutôt qu'un SLAM approximatif

9. Structure de la Documentation Finale

9.1 Rapport de Recherche (à remettre)

- Chapitre 1 — Introduction & Contexte (réutiliser l'état de l'art)
- Chapitre 2 — Architecture & Spécifications (basé sur ce document)
- Chapitre 3 — Implémentation de la Solution 2 (fingerprinting + PF)
- Chapitre 4 — Implémentation de la Solution 1 (Wi-Fi RTT + SLAM si réalisé)
- Chapitre 5 — Résultats expérimentaux & Analyse comparative
- Chapitre 6 — Discussion, Limites & Perspectives
- Annexes : code source commenté, dataset fingerprinting, paramètres de calibration

9.2 Livrables Techniques

- Code source complet sur GitHub (dépôt public ou privé selon directive des encadrants)
- APK Android installable de l'application finale
- Dataset fingerprinting (JSON/CSV) de l'espace test
- Fichiers de calibration caméra (XML OpenCV)
- Serveur Flask (code Python + instructions de déploiement)
- Vidéo de démonstration (2–3 min) : démarrage app → alerte → navigation → arrivée